Two levels, and what each one actually protects

Chosen per vault at creation. The difference is confidentiality alone — integrity is not a tier and has no off switch at either level.

What is written to disk

	Sealed — the default	HmacOnly
drawer content	zstd, then encrypted	plaintext
embedding i8-quantized first — a vector of the text leaks the text	encrypted	in clear
ColBERT token matrices	encrypted	in clear
PQ codes, codebook, pages	encrypted	in clear
full-text (fts5) index	never exists	built, and used as a prefilter
drawer metadata (meta_json)	UNSEALED	UNSEALED
per-row HMAC tag	always — verified on every read	always
audit chain entry	always — same transaction	always

The standing rule for a sealed vault

No plaintext, and nothing derived from plaintext, is written to disk in clear. Search runs over decrypt-once RAM caches, and any new derived artifact has to follow the same pattern.

When HmacOnly is the right answer

The database stays readable with ordinary SQLite tools and the lexical prefilter exists, which matters at scale. You are trading confidentiality for inspectability — not integrity, which stays.

What an attacker holding a SEALED database file can still read

Measured against the actual bytes, not asserted. meta_json is stored unsealed so it can be filtered in SQL, and these fields sit inside it:

```
wing · room · source_file · added_by · hall · content_date · the dates resolved out of the content
```

They read no word of the content itself. But in practice a wing is a person or a case, a room is a topic, and a source path carries a username and a project — so until this closes, treat those names as public labels and keep the secret out of the name.

Pinned by a test that fails in BOTH directions: it fails if content ever leaks, and it fails if a listed field stops being readable — so shrinking the exposure forces the inventory to be updated rather than letting it quietly go stale.

Closing it does not mean giving up queryability — both halves already exist in the engine

Fields that must be filtered in SQL (wing, room) become a keyed blind index — store a truncated HMAC and filter on that, exactly as duplicate detection already does with its keyed fingerprint. Everything else moves into a sealed blob with a RAM cache, exactly as embeddings, PQ codes and ColBERT matrices already are. The pattern for having both is in use four times over.